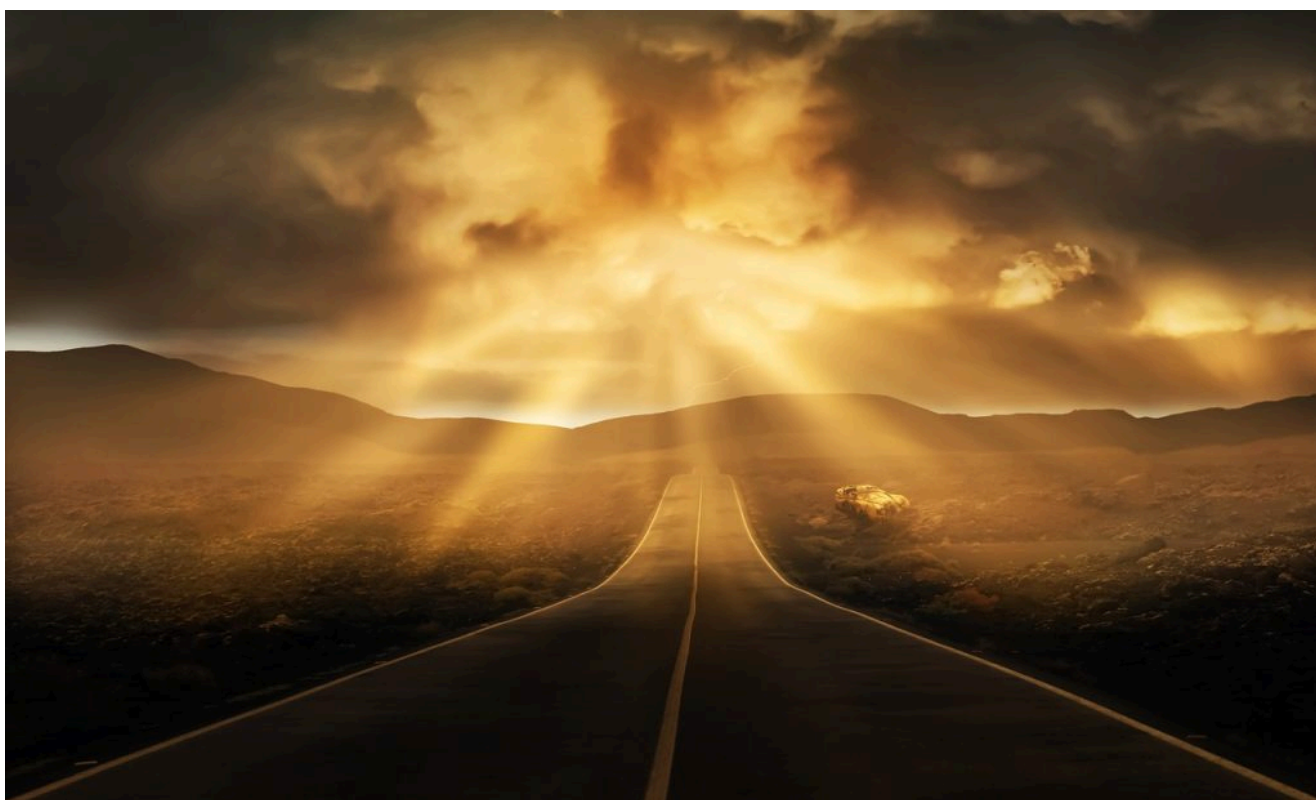


CLOUD NATIVE ECOSYSTEM / КОНТЕЙНЕРЫ / KUBERNETES

10 ключевых характеристик облачных нативных приложений

Что такое Cloud Native-вычисления? Какие инструменты и платформы нужны для полного переноса рабочих нагрузок в облако? Все ответы вы найдете здесь.

19 июля 2018 года, 9:00 [Джанакирам MSV](#)



Изображение для заставки [через Pixabay](#).

Cloud native — термин, используемый для описания контейнерных сред. Cloud native-технологии используются для разработки приложений, состоящих из сервисов, упакованных в контейнеры, развернутых как микросервисы и управляемых в эластичной инфраструктуре с помощью гибких процессов DevOps и рабочих процессов непрерывной поставки.

В то время как операционные команды распределяли ресурсы инфраструктуры для традиционных приложений вручную, облачные приложения развертываются в инфраструктуре, которая абстрагируется от базовых вычислительных ресурсов, хранилищ и сетевых примитивов.

(API), предоставляемыми поставщиками инфраструктуры. Вместо этого оркестратор автоматически распределяет ресурсы в соответствии с политиками, установленными командами DevOps. Контроллер и планировщик, которые являются важнейшими компонентами оркестратора, управляют распределением ресурсов и жизненным циклом приложений.

Облачные нативные платформы, такие как **Kubernetes**, предоставляют единую сеть, которая накладывается на существующие сетевые топологии и примитивы облачных провайдеров. Аналогичным образом нативный уровень хранения данных часто абстрагируется для предоставления логических томов, интегрированных с контейнерами. Операторы могут выделять квоты на хранение данных и сетевые политики, к которым имеют доступ разработчики и администраторы ресурсов. Абстракция инфраструктуры не только обеспечивает переносимость между облачными средами, но и позволяет разработчикам использовать новые подходы для создания и развертывания приложений. Менеджеры оркестрации становятся целью развертывания, независимо от базовой инфраструктуры, которая может быть основана на физических серверах или виртуальных машинах, частных или публичных облаках.

Kubernetes — идеальная платформа для современных рабочих нагрузок, разработанных как **облачные нативные приложения**. Она стала де-факто операционной системой для облака, во многом подобно тому, как Linux является операционной системой для базовых машин. Если разработчики будут следовать лучшим практикам проектирования и разработки программного обеспечения в виде набора микросервисов, составляющих облачные нативные приложения, команды DevOps смогут упаковывать и развертывать их в Kubernetes. Вот 10 ключевых характеристик облачных нативных приложений, о которых разработчикам следует помнить при их проектировании.

[cycloneslider id="kubernetes-series-book-3-sponsors"]

10 Key Attributes of Cloud Native Applications

1. **Packaged as lightweight containers:** **Cloud native applications** are a collection of independent and autonomous services that are packaged as lightweight containers. Unlike virtual machines, containers can scale-out and scale-in rapidly. Since the unit of scaling shifts to containers, infrastructure utilization is optimized.

variety of languages, runtimes and frameworks. For example, developers may build a real-time streaming service based on WebSockets, developed in Node.js, while choosing Python and Flask for exposing the API. The fine-grained approach to developing microservices lets them choose the best language and framework for a specific job.

3. **Designed as loosely coupled microservices:** Services that belong to the same application discover each other through the application runtime. They exist independent of other services. Elastic infrastructure and application architectures, when integrated correctly, can be scaled-out with efficiency and high performance.

Loosely coupled services allow developers to treat each service independent of the other. With this decoupling, a developer can focus on the core functionality of each service to deliver fine-grained functionality. This approach leads to efficient lifecycle management of the overall application, because each service is maintained independently and with clear ownership.

4. **Centered around APIs for interaction and collaboration:** Cloud native services use lightweight APIs that are based on protocols such as representational state transfer (REST), Google's open source remote procedure call (gRPC) or NATS. REST is used as the lowest common denominator to expose APIs over hypertext transfer protocol (HTTP). For performance, gRPC is typically used for internal communication among services. NATS has publish-subscribe features which enable asynchronous communication within the application.
5. **Architected with a clean separation of stateless and stateful services:** Services that are persistent and durable follow a different pattern that assures higher availability and resiliency. Stateless services exist independent of stateful services. There is a connection here to how storage plays into container usage. Persistence is a factor that has to be increasingly viewed in context with state, statelessness and — some would argue — micro-storage environments.
6. **Isolated from server and operating system dependencies:** Cloud native applications don't have an affinity for any particular operating system or individual machine. They operate at a higher abstraction level. The only exception is when a microservice needs certain capabilities, including solid-state drives (SSDs) and graphics processing units (GPUs), that may be exclusively offered by a subset of machines.
7. **Deployed on self-service, elastic, cloud infrastructure:** Cloud native applications are deployed on virtual, shared and elastic infrastructure. They may align with the underlying infrastructure to dynamically grow and shrink — adjusting themselves to the varying load.
- . **Managed through agile DevOps processes:** Each service of a cloud native application goes through an independent life cycle, which is managed through an

9. **Automated capabilities:** Cloud native applications can be highly automated. They play well with the concept of infrastructure as code. Indeed, a certain level of automation is required simply to manage these large and complex applications.
10. **Defined, policy-driven resource allocation:** Finally, cloud native applications align with the governance model defined through a set of policies. They adhere to policies such as central processing unit (CPU) and storage quotas, and network policies that allocate resources to services. For example, in an enterprise scenario, central IT can define policies to allocate resources for each department. Developers and DevOps teams in each department have complete access and ownership to their share of resources.

TRENDING STORIES

1. [The Bakery Model for Building Container Images and Microservices](#)
2. [10 Key Attributes of Cloud Native Applications](#)
3. [5 Tools for Monitoring Kubernetes at Scale in Production](#)
4. [Managing Persistence for Docker Containers](#)
5. [The Container Networking Landscape: CNI from CoreOS and CNM from Docker](#)

TNS



Janakiram MSV (Jani) is a practicing architect, research analyst, and advisor to Silicon Valley startups. He focuses on the convergence of modern infrastructure powered by cloud-native technology and machine intelligence driven by generative AI. Before becoming an entrepreneur, he spent...

[Read more from Janakiram MSV →](#)

SHARE THIS STORY



3. 5 Tools for Monitoring Kubernetes at Scale in Production
4. Managing Persistence for Docker Containers
5. The Container Networking Landscape: CNI from CoreOS and CNM from Docker

INSIGHTS FROM OUR SPONSORS

[Research: How we cut AI costs by 80%](#)

25 June 2026

[The Agentic SDLC: The Software Lifecycle, Rebuilt Around Agents](#)

22 June 2026

[The Hidden Technical Debt Of Agentic Engineering](#)

20 June 2026

[Build Azure Infrastructure Using AI and Terraform](#)

11 June 2025

[GenAI Made Terraform More Relevant Than Ever](#)

1 April 2025

[The Deployment Bottleneck No One Talks About](#)

26 March 2025

[Hydrolix Awards: One of Fastest-Growing Companies](#)

regon

une 2026

The Data Your Next Investigation Needs Is Being Deleted Right Now

16 June 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
JMX

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development

Networking
Storage

Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

 roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



© The New Stack 2026

Disclosures
Terms of Use
Advertising Terms & Conditions
Privacy Policy
Cookie Policy

